

SOK-1305 Statistisk læring for økonomer

Kapittel 8.2: Bagging, Random Forest og Boosting

Ensemble-metoder: Mange "weak learners" blir sterke sammen!

Forelesningsnotater — Handelshøgskolen, UiT

"Sampling"

"rebasende"

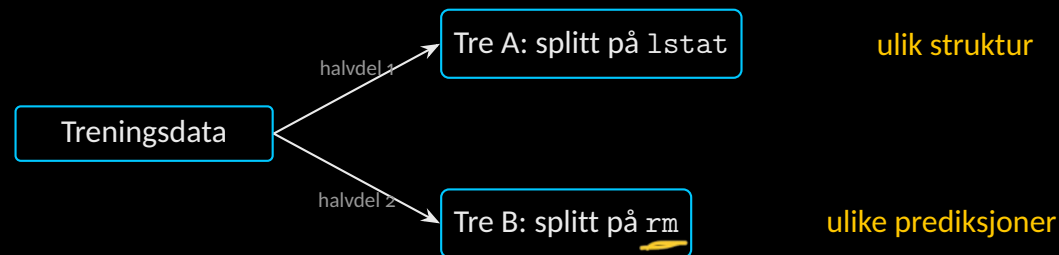
Agenda

1. Motivasjon: hvorfor kombinere mange modeller?
2. Bootstrap i bunn: hvordan lage B datasett av ett. — i kgs.
3. **Bagging** — redusere varians.
4. **Out-of-Bag (OOB)** — «gratis» testfeil.
5. **Variabelviktighet** — tolkning uten å se treet. — "variable importance"
6. **Random Forest** — dekorrelering av trær via m .
7. **Boosting** — å lære sakte av egne feil.
8. Sammenligning, kode og overgang til SVM.

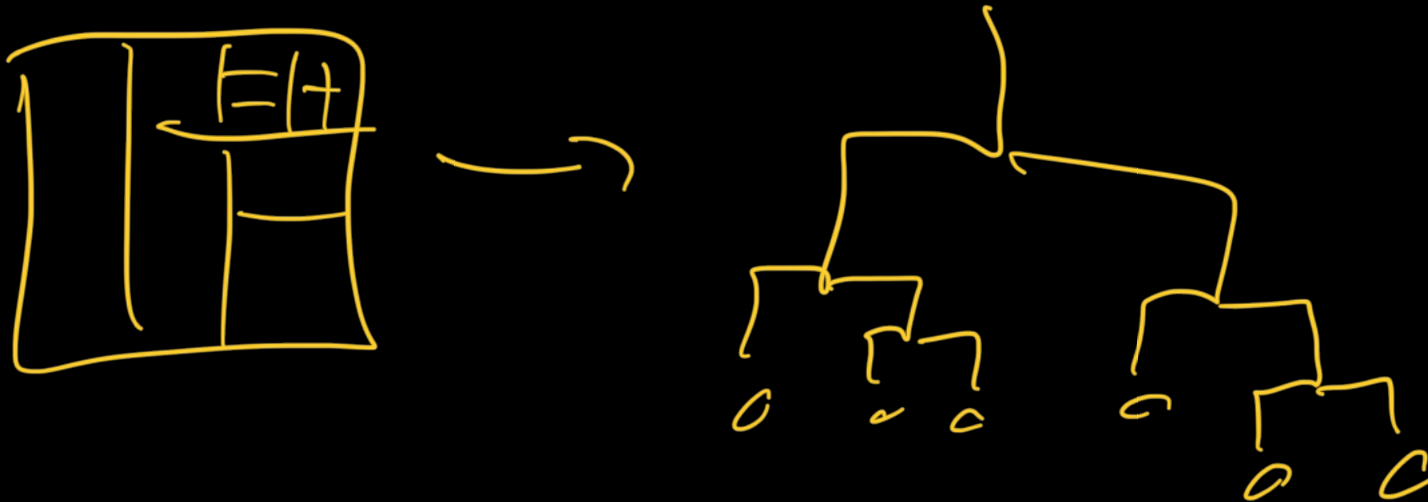
Kap 2. Bias Variance Trade off.

Repetisjon fra 8.1: ett tre er ustabil

Et dypt beslutningstre har lav bias (det kan tilpasse seg nesten hva som helst), men høy varians: deler vi treningsdataene tilfeldig i to og estimerer ett tre på hver halvdel, kan vi få to helt ulike trær.



Et lite skifte i data $\Rightarrow$ stort skifte i modell. Det er ikke tilfeldig støy vi vil ha — det er *estimeringsvariens*. Hele kapittel 8.2 handler om å fjerne denne variansen uten å gi slipp på den lave biasen.



Kjerneidéen: gjennomsnitt gir lavere varians

For n uavhengige observasjoner $Z_1, \dots, Z_n$ med varians σ^2 :

$$\text{Var}(\bar{Z}) = \frac{\sigma^2}{n} \quad \leftarrow \text{antall obs.}$$

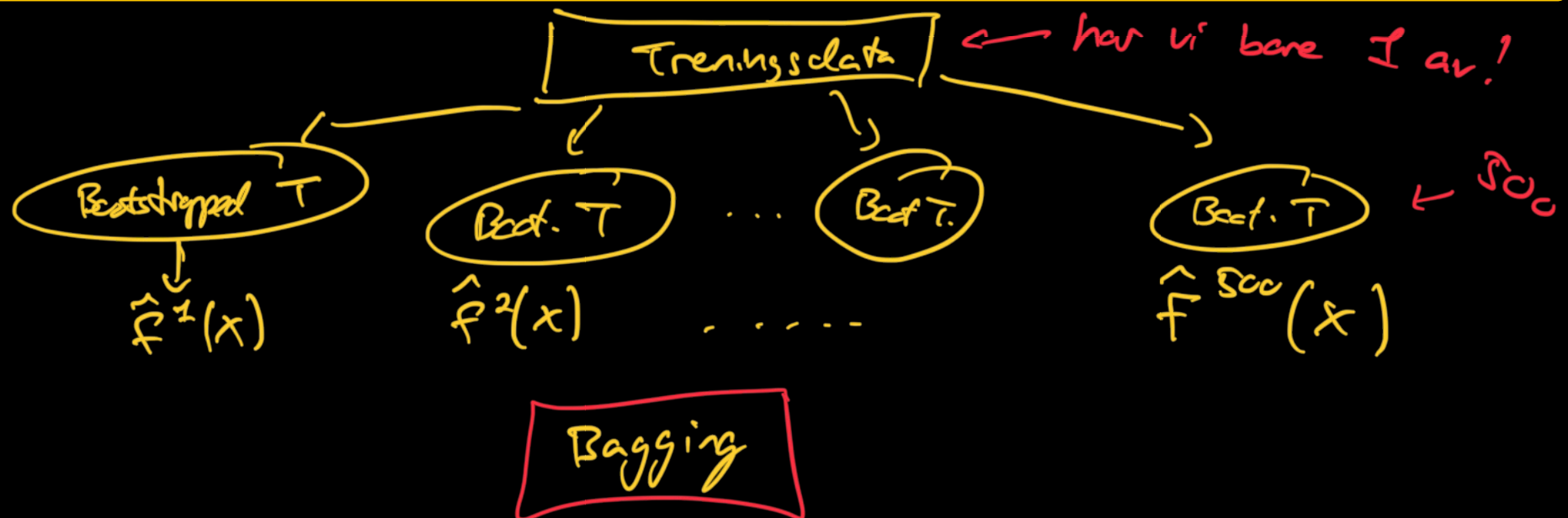
Overført til modeller: hadde vi hatt B uavhengige treningssett kunne vi estimert $\hat{f}^1(x), \dots, \hat{f}^B(x)$ og tatt gjennomsnittet

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^b(x),$$

som ville hatt samme bias, men $1/B$ av variansen.

Viktig: Problemet

Vi har som regel ett datasett. Vi har ikke råd til å samle inn 500 nye utvalg av norske husholdninger. Hva er løsningen?



Bootstrap: trekking med tilbakelegging

Fra ett utvalg på n observasjoner trekker vi n observasjoner **med tilbakelegging**, B ganger. Noen observasjoner kommer med flere ganger (skyggelagt), andre ikke i det hele tatt (gult).



Hvert bootstrap-utvalg har like mange observasjoner som originalen, men ulik sammensetning. Derfor blir treet vi estimerer på hvert utvalg litt forskjellig — og *det* er variasjonen vi vil skal minke.

$\hat{f}(x) \rightarrow$ tester på $z_3 \rightarrow MSE$
tester på $z_5 \rightarrow MSE$



Hvor mye av dataene havner i sekken?

Sannsynligheten for at observasjon i ikke trekkes i én bestemt trekning er $1 - \frac{1}{n}$. Trekker vi n ganger uavhengig:

$$P(\text{obs } i \text{ er ute av utvalget}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0,368. \approx 1/3$$

n	5	10	25	100	500	1000
$P(\text{ute}) = (1 - 1/n)^n$	0,328	0,349	0,360	0,366	0,368	0,368
$P(\text{i sekken})$	0,672	0,651	0,640	0,634	0,633	0,632

Viktig: Tommelfingerregelen

Hvert bootstrap-utvalg inneholder ca. $2/3$ av de unike observasjonene. Den siste $1/3$ er «out-of-bag» for det treet — og den skal vi gjenbruke gratis om noen sider.

Notater

Blank notepad area for taking notes.

Algoritme: Bagging (Bootstrap Aggregating)

1. Lag B bootstrap-utvalg fra treningsdataene.
2. Estimer et **dypt, ubeskåret** tre $\hat{f}^{*b}(x)$ på hvert utvalg, $b = 1, \dots, B$.
3. Kombiner:

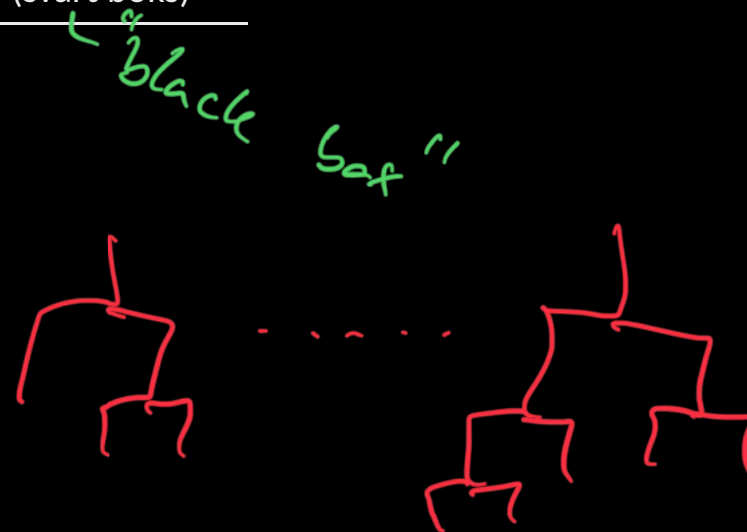
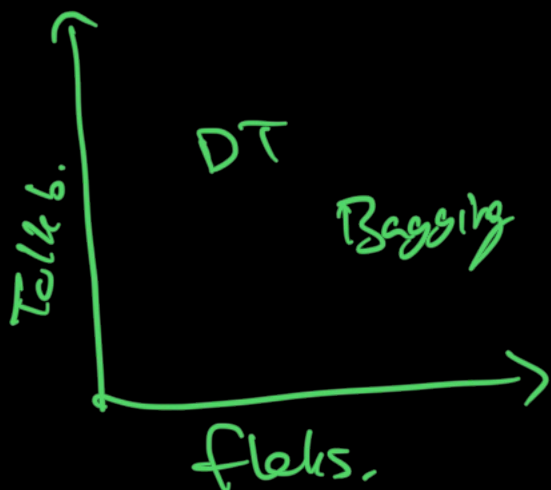
- **Regresjon:** $\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$

- **Klassifisering:** flertallsavstemning over de B predikerte klassene.

majority vote

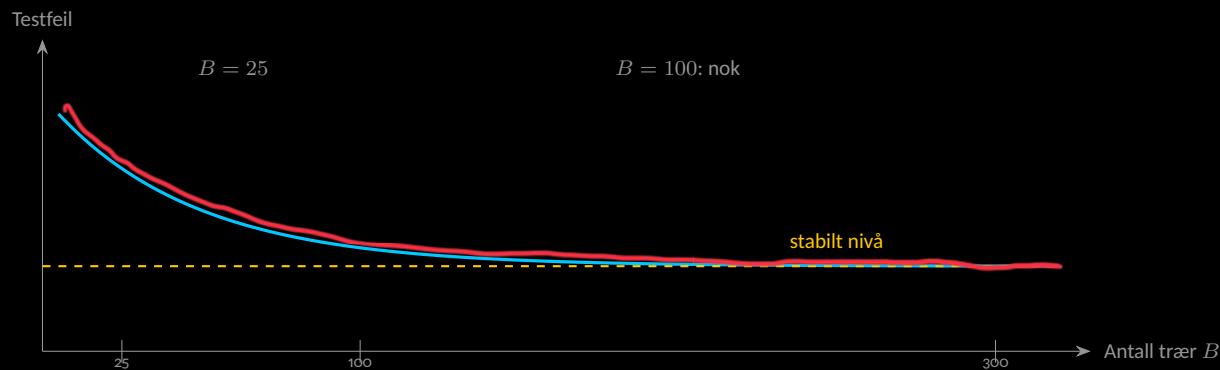
Hvorfor dype trær? Vi vil bevisst ha lav bias i hvert enkelt tre. Beskjæring (pruning) er et *varians*-verktøy — og varians tar gjennomsnittet seg av. Å beskjære hvert tre i bagging ville bare lagt til bias uten gevinst.

	Ett dypt tre	Bagging med B trær
Bias	Lav	Lav (uendret)
Varians	Høy	Betydelig lavere
Tolkbarhet	Høy (kan tegnes)	Lav (svart boks)



Hvor mange trær? B er ikke en tuningparameter

Bagging (og Random Forest) **overtilpasser ikke** når B øker. Flere trær gir bare et mer presist gjennomsnitt — feilen flater ut og blir liggende.



Praktisk råd: velg B stort nok til at kurven har flatet ut. I ISLP holder $B = 100$ for *Heart*-dataene; med mange prediktorer er $B = 500$ en trygg standard. Ulempen ved stor B er compute, ikke overtilpasning.

Eksempel: Boston — prediksjon av medianboligpris med v

$n = 506$ forsteder, $p = 12$ prediktorer, 70/30 trening/test-splitt (ISLP-laben).

Modell	Test-MSE	Kommentar
Ett beskåret tre (CV-valgt α)	28,07	$\sqrt{\text{MSE}} \approx 5,30 \Rightarrow \pm \$5\,300$
Bagging, $B = 100$ ($m = p = 12$)	14,63	omtrent <i>halvert</i> feil
Bagging, $B = 500$	14,61	nesten ingen endring — kurven har flatet ut

Viktig: Økonomisk lesning

Standardavviket i prediksjonsfeil faller fra ca. \$5 300 til ca. $\sqrt{14,63} \approx 3,83$, altså ca. \$3 800 per forstad. Prisen vi betaler er at vi ikke lenger kan tegne modellen på tavla og vise klienten beslutningsveien.

Out-of-Bag: testsettet vi får gratis

Observasjon i er *out of bag* i ca. $1/3$ av trærne. De trærne har *aldri sett* i under trening — akkurat som et testsett.

Obs.	Tre 1	Tre 2	Tre 3	Tre 4	Tre 5
z_1	i sekken	i sekken	i sekken	i sekken	OOB
z_2	i sekken	i sekken	OOB	i sekken	i sekken
z_3	i sekken	OOB	OOB	i sekken	i sekken
z_4	OOB	i sekken	i sekken	OOB	i sekken
z_5	i sekken	OOB	i sekken	i sekken	OOB
z_6	i sekken	i sekken	i sekken	OOB	i sekken

$\hat{y}_4^{\text{OOB}} = \frac{1}{2} \left(\hat{f}^1(x_4) + \hat{f}^4(x_4) \right)$

Med $B = 5$ er dette grovt; med $B = 500$ får hver observasjon ca. $B/3 \approx 167$ uavhengige prediksjoner å testes på.

Algoritme: OOB-feilestimat

1. For hver observasjon i : finn de trærne der i var out-of-bag (ca. $B/3$ stk.).
2. Midle deres prediksjoner (regresjon) eller ta flertallsavstemning (klassifisering) $\Rightarrow$ én OOB-prediksjon $\hat{y}_i^{\text{OOB}}$.
3. Regn ut samlet feil over alle n observasjoner:

$$\text{OOB-MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i^{\text{OOB}})^2 \quad \text{eller} \quad \text{OOB-feilrate} = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i^{\text{OOB}}).$$

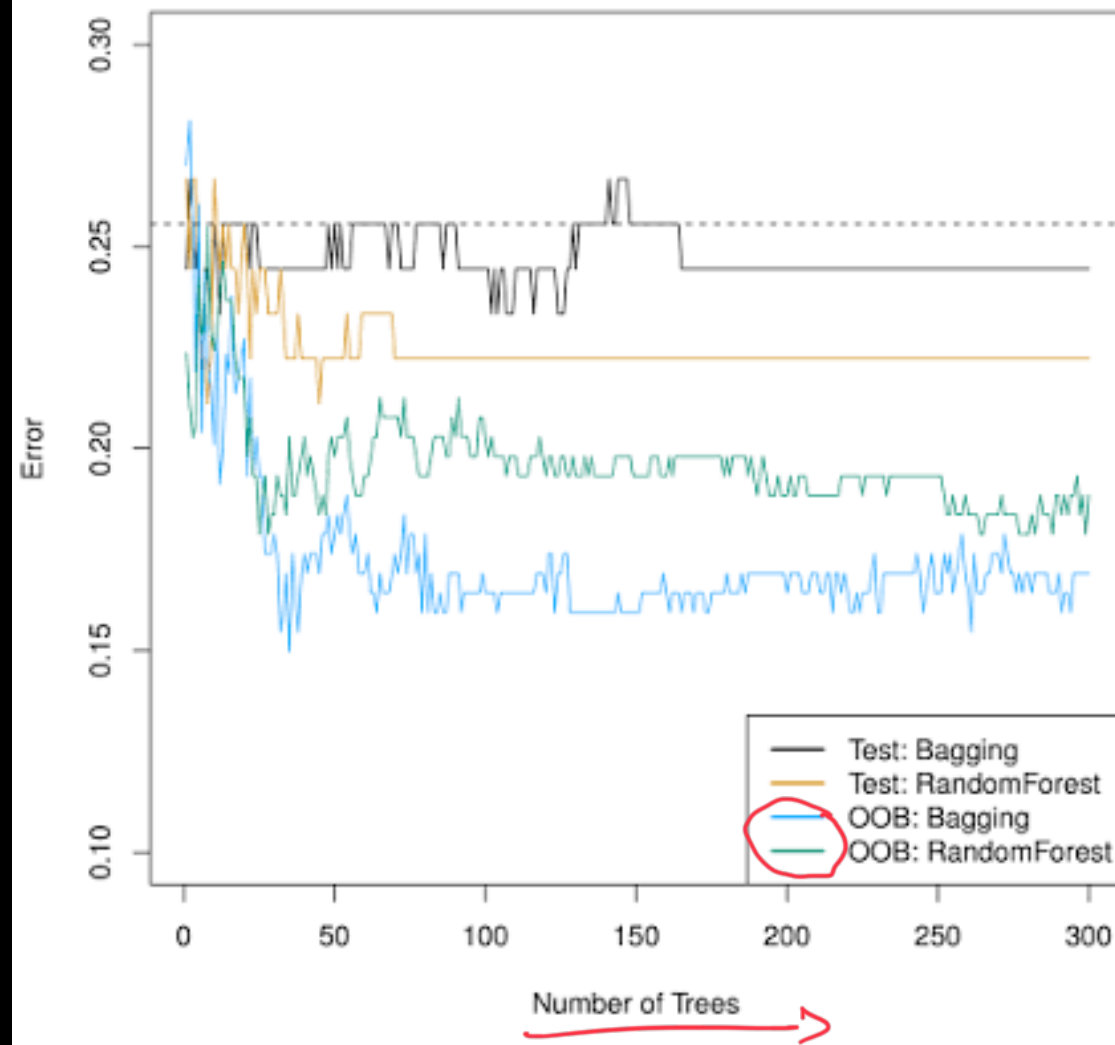
regresjon

klassifisering

Viktig: Hvorfor dette er stort

OOB-feilen er et gyldig estimat på testfeilen, og den koster **ingenting ekstra** å regne ut. Med 10-folds kryssvalidering må vi estimere hele ensemblet 10 ganger. Med OOB holder det med én runde. For store B er OOB-feilen tilnærmet lik LOOCV-feilen for den baggede modellen.

Merk: OOB er et estimat, ikke en fasit. I ISLP figur 8.8 ligger OOB-kurven for *Heart*-dataene tilfeldigvis *under* testfeilen — forfatterne påpeker eksplisitt at dette er tilfeldig.



Figur 1: Figur 8.8 fra ISLP boka

"Variable importance"

Variabelviktighet: tolkning uten å se treet

Vi mister det tegnbare treet, men ikke all innsikt. For hver prediktor summerer vi hvor mye splitter på den variabelen reduserte urenheten, og ser på det over alle B trær.

• Regresjonstrær: gjennomsnittlig reduksjon i RSS.

• Klassifikasjonstrær: gjennomsnittlig reduksjon i Gini-indeksen.

- Tallene rapporteres ofte relativt til den største (%), eller normalisert til å summere til 1.

← Sci.kit. learn

Eksempel: Boston — feature_importances_ fra Random Forest

lstat	0,369	indus	0,053	age	0,024
rm	0,334	crim	0,052	tax	0,022
ptratio	0,057	dis	0,042	rad, zn, chas	< 0,01

Sosioøkonomisk status i nabolaget (lstat) og boligstørrelse (rm) står for ca. **70 %** av den samlede reduksjonen i urenheter.

Les viktighetsmål med økonomisk kontekst

Variabelviktighet (eng: *variable importance*) er et prediksjonsmål, ikke et kausalt mål.

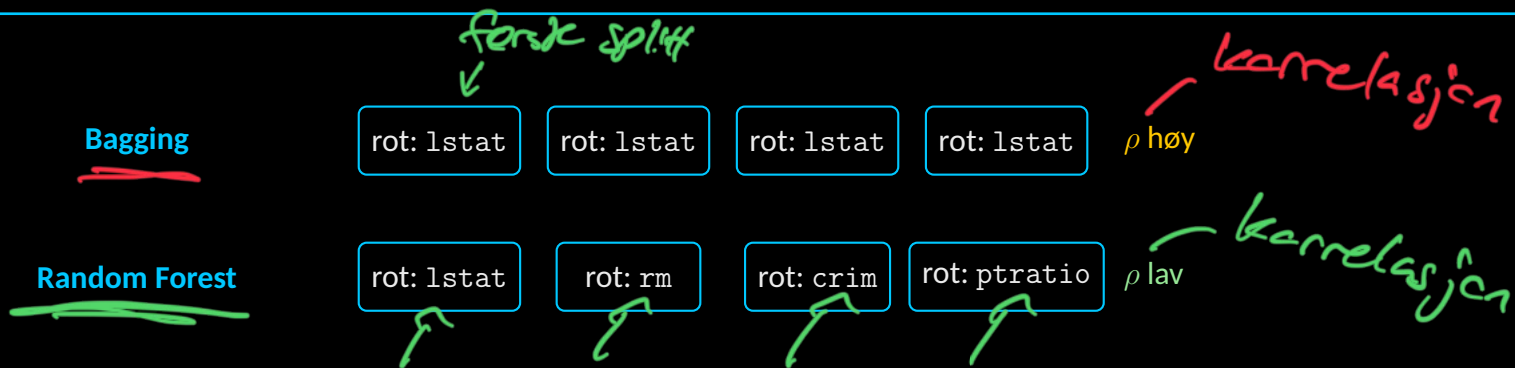
- En variabel kan skåre høyt fordi den er en god proxy for noe annet. Høy viktighet for lstat betyr ikke at et tiltak mot fattigdom flytter boligprisen med en bestemt sum.
- Målet er relativt: rekkefølgen er informativ, nivåtallene i seg selv er det knapt.
- Måten er skjev i favør av variabler med mange mulige splittepunkter (kontinuerlige variabler og kategoriske variabler med mange nivåer får flere sjanser til å 'se bra ut').
- To sterkt korrelerte prediktorer «deler» viktigheten mellom seg — begge kan se middelmådige ut selv om informasjonen de bærer er avgjørende.

Viktig: Bruk

Bruk plots fra variable importance til å stille spørsmål «hvorfor er p_{ratio} viktigere enn crim?», ikke til å konkludere om årsakssammenhenger.

Svakheten i bagging: trærne ligner hverandre

Finnes det én dominerende prediktor, vil *nesten alle* trærne velge den i toppsplitten — uansett hvilket bootstrap-utvalg de fikk.

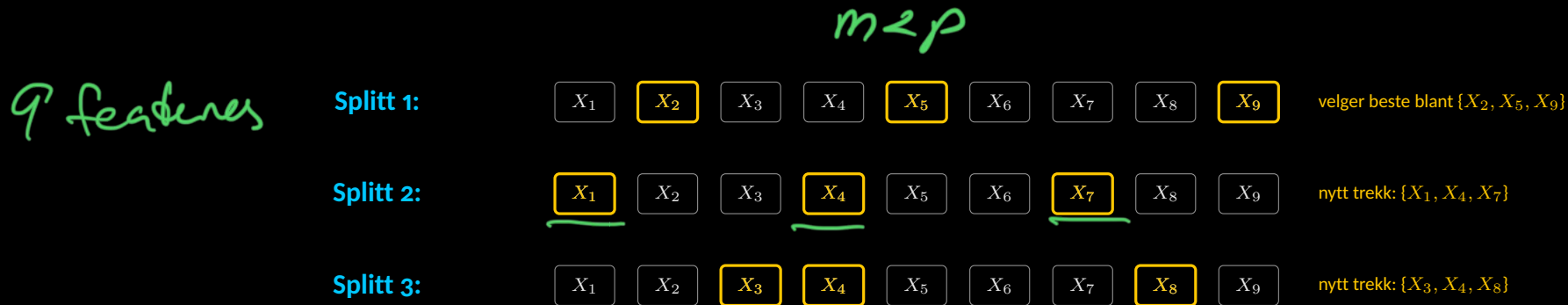


Gjennomsnittet av B korrelerte størrelser reduserer ikke variansen like mye som gjennomsnittet av B uavhengige. Bagging løser dermed bare halve problemet: vi har B trær, men i praksis langt færre enn B uavhengige meninger.

bedre med
treningssett.

Random Forest: begrenns utvalget ved hver splitt

Som i bagging bygger vi B trær på bootstrap-utvalg. Forskjellen: hver gang en splitt skal gjøres, får algoritmen kun velge blant m tilfeldig trukne prediktorer av de p tilgjengelige. Et nytt utvalg trekkes ved hver eneste splitt — ikke én gang per tre.



Her er $p = 9$ og $m = 3$. Den sterke prediktoren er utelatt fra vurderingen i $(p - m)/p = 6/9 \approx 67\%$ av splittene — og da må treet finne signal et annet sted.

m er hele forskjellen mellom bagging og Random Forest

$$m = p \implies \text{bagging}$$

$$m < p \implies \text{Random Forest}$$

Bagging er altså *spesialtilfellet* $m = p$. Det er derfor RandomForestRegressor kan gjøre begge deler.

p	$m = p$ (bagging)	$m \approx \sqrt{p}$	Andel splitter uten sterk prediktor $(p - m)/p$
4	4	2	50,0 %
9	9	3	66,7 %
13	13	4	69,2 %
100	100	10	90,0 %
500	500	22	95,6 %

Viktig: Standardvalg

$m \approx \sqrt{p}$ for klassifisering (13 prediktorer i *Heart* $\Rightarrow m = 4$), og $m \approx p/3$ er vanlig for regresjon. I scikit-learn er `max_features` som standard *alle* prediktorene for RandomForestRegressor (altså bagging!) og `'sqrt'` for RandomForestClassifier. Sett den derfor eksplisitt — og behandle m som en tuningparameter du velger med OOB eller kryssvalidering.

Hvis vi har 16 prediktorer $\rightarrow m = 4$

Regnestykket bak dekorrelering

Er B størrelser hver med varians σ^2 og parvis korrelasjon ρ , er variansen til gjennomsnittet

$$\text{Var}(\bar{X}) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2 \xrightarrow{B \rightarrow \infty} \rho \sigma^2.$$

Korrelasjon ρ	$\text{Var}(\bar{X})$ ved $B = 500, \sigma^2 = 1$	Gulv når $B \rightarrow \infty$
0,90 (bagging, sterk prediktor)	0,9002	0,90
0,60	0,6008	0,60
0,30 (Random Forest)	0,3014	0,30
0,00 (uavhengige trær)	0,0020	0,00

Poenget: å øke B fjerner bare det andre leddet. Første ledd, $\rho \sigma^2$, er et 'gulv' som flere trær ikke kommer under. Random Forest angriper nettopp dette 'gulvet' ved å senke ρ . Det er derfor et lite m hjelper mest når vi har mange og sterkt korrelerte prediktorer.

(Dette uttrykket står ikke i ISLP — det er tatt med her som intuisjon. Til eksamen holder det å forklare ideen i ord.)

Eksempel: Når hjelper Random Forest — og når gjør den det ikke?

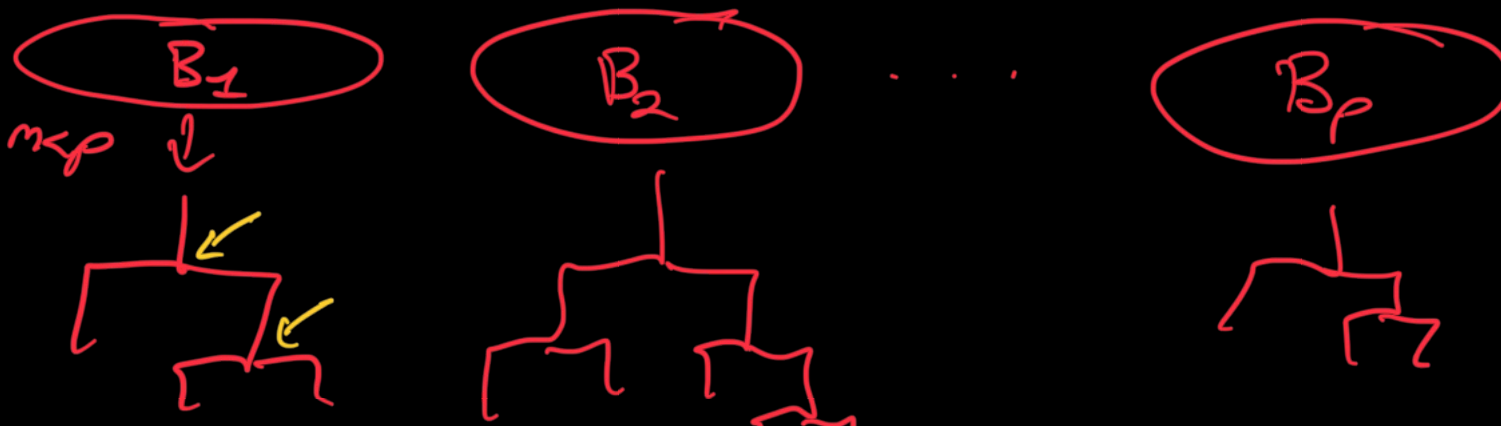
Datasekk	Modell	Testfeil
Boston ($p = 12$, regresjon)	Bagging ($m = 12$)	MSE 14,63
Boston	Random Forest ($m = 6$)	MSE 20,04
Genuttrykk ($p = 500$, 15 klasser)	Ett tre	45,7 % feil
Genuttrykk	Bagging ($m = p$)	ca. 30 % feil
Genuttrykk	Random Forest ($m = \sqrt{p}$)	litt lavere enn bagging

Viktig: Ærlig lesning av tallene

På Boston gjorde Random Forest det **dårligere** enn bagging. Med bare 12 prediktorer, der to av dem (lstat, rm) bærer nesten alt signalet, koster det mer å utestenge dem fra to tredjedeler av splittene enn vi vinner på dekorrelering. Med $p = 500$ snur bildet.

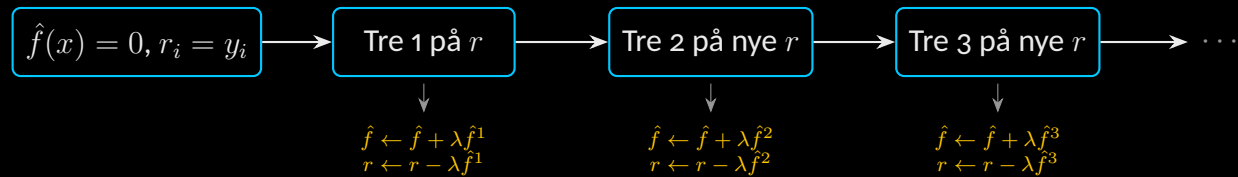
Regel: jo større p og jo mer korrelerte prediktorene er, desto mer lønner et lite m seg. «Random Forest slår alltid bagging» er feil — m må velges, ikke antas.

er clones
j = 66.



Boosting: sekvensielt, ikke parallelt

Bagging og Random Forest bygger trærne uavhengig og midler til slutt. Boosting bygger trærne sekvensielt: hvert nytt tre estimeres på det forrige treets feil. Ingen bootstrap — hvert tre ser hele datasettet, men med oppdatert responsvariabel.



Hvert tre er lite (ofte bare noen få splitter). Alene er det en elendig modell — en *weak learner*. Summen av mange små korrek-sjoner blir likevel svært presis.

$$\lambda\text{-vec} = [0.01, 0.05, \dots]$$

Algoritme: Boosting for regresjonstrær (ISLP 8.2)

1. Sett $\hat{f}(x) = 0$ og $r_i = y_i$ for alle i i treningssettet.

2. For $b = 1, 2, \dots, B$: ← 5000

(a) Estimer et tre $\hat{f}^b$ med d splitter ($d + 1$ terminalnoder) på dataene (X, r) .

(b) Oppdater modellen med en krympet (shrunk) versjon av det nye treet:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \cdot \hat{f}^b(x) \quad \leftarrow \lambda = 0.01$$

(c) Oppdater residualene:

diff nye

gammelt tre

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i)$$

3. Returner den boostede modellen:

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x).$$

Viktig: Merk responsen

I steg 2(a) er responsvariabelen residualen, ikke y . Vi spør hele tiden: «hvor tar modellen fortsatt feil, og finnes det en enkel regel som fanger den feilen?»

tuning parametere

De tre parametere i boosting

I motsetning til Random Forest krever boosting en del mer *tuning*.

1. **Antall trær B .** Her kan for mange trær gi overtilpasning — ulikt bagging og Random Forest. Overtilpasningen kommer sakte, men den kommer. Velg B med kryssvalidering.
2. **Krympeparameteren λ (læringsraten).** Typiske verdier er 0,01 eller 0,001. Liten λ betyr at hvert tre bidrar lite $\Rightarrow$ trenger stor B . λ og B må velges sammen: halverer du λ , må B omtrent dobles.
3. **Antall splitter d (interaksjonsdybden).** $d = 1$ gir stubber, og summen av stubber er en **additiv modell** $f(X) = \sum_{j=1}^p f_j(X_j)$ — ingen interaksjoner. $d = 2$ tillater toveis interaksjoner, osv. Ofte holder $d = 1$ eller $d = 2$.

Må man ha et
bestemt forhold til
hvor vi koder.
Kryssvalidering!

Eksempel: Boston — boosting med GradientBoostingRegressor

Samme 70/30-splitt som før, $B = 5000$ trær, $\text{max_depth} = 3$.

Modell	Test-MSE
Ett beskåret tre	28,07
→ Bagging ($m = p = 12$, $B = 100$)	14,63
Random Forest ($m = 6$)	20,04
Boosting ($\lambda = 0,001$, $d = 3$, $B = 5000$)	14,48
Boosting ($\lambda = 0,2$, $d = 3$, $B = 5000$)	14,50

Viktig:

Boosting vinner her — men med hårfin margin over bagging (14,48 mot 14,63), og forskjellen er godt innenfor det som ville variert med et annet tilfeldig splitt av data. På ett datasett med én splitt kan vi *ikke* rangere metodene med sikkerhet. Det samme gjelder i ISLPs genuttrykkseksempel: stubber slår $d = 2$ og Random Forest, men standardfeilene er rundt 0,02, så ingen av forskjellene er statistisk meningsfulle.

Hvilken metode velger jeg?

eller de?

Egenskap	Bagging	Random Forest	Boosting
Byggestil	Parallell	Parallell	Sekvensiell
Bootstrap?	Ja	Ja	Nei
Trærnes dybde	Dype	Dype	Svært grunne ($d = 1-3$)
Responsvariabel	y	y	Residualene r
Nøkkelparameter	(ingen) $m=p$	$m < p$	B, λ, d
Hovedmål	Redusere varians	Redusere korrelasjon	Redusere bias
Overtilpasser ved stor B ?	Nei	Nei	Ja, sakte
Gratis testfeil (OOB)?	Ja	Ja	Nei (bruk CV)

- + Høy prediksjonskraft, robust mot uteliggere, trenger ingen skalering av variabler.
- + Håndterer kvalitative prediktorer og ikke-lineære sammenhenger uten spesifisering fra oss.
- Svart boks: vi mister den tegnbare beslutningsveien.
- Beregningstungt, og boosting krever tuning for å fungere godt.

ISLP 8.2.4 omtaler i tillegg **BART**, som kombinerer tilfeldighet (som RF) med residualtilpasning (som boosting). På Boston gir BART test-MSE 20,92. Vi nevner den, men går ikke i dybden.

Koden i scikit-learn

Bagging og Random Forest er samme funksjon — forskjellen er max_features.

✓ sklearn og stat models i Python

```
from sklearn.ensemble import RandomForestRegressor as RF
from sklearn.ensemble import GradientBoostingRegressor as GBR
```

```
# Bagging: alle p prediktorer vurderes ved hver splitt
bag = RF(max_features=X_train.shape[1], n_estimators=500,
         oob_score=True, random_state=0).fit(X_train, y_train)
np.mean((y_test - bag.predict(X_test))**2) # 14.61
bag.oob_score_ # "gratis" validering
```

```
# Random Forest: kun m av p prediktorer ved hver splitt
rf = RF(max_features=6, n_estimators=500, random_state=0).fit(X_train, y_train)
```

$m=6 \ll p$

```
# Variabelviktighet
pd.Series(rf.feature_importances_, index=feature_names).sort_values(ascending=False)
```

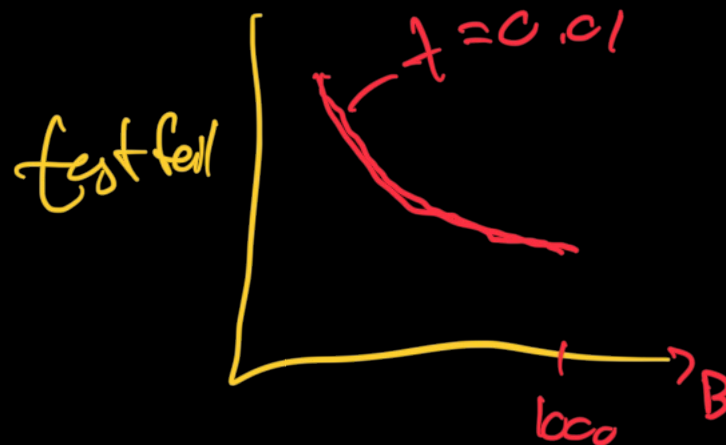
```
# Boosting: lambda = learning_rate, d = max_depth, B = n_estimators
boost = GBR(n_estimators=5000, learning_rate=0.001, max_depth=3,
            random_state=0).fit(X_train, y_train)
```

oob_score=True må slås på eksplisitt. Klassifisering: bytt til RandomForestClassifier / GradientBoostingClassifier.

Oppgaver for diskusjon

Diskuter i gruppe. Ingen kode — penn og papir.

- z_3 mangler.
- "tydelig p^2 "
1. Et bootstrap-utvalg fra $n = 4$ observasjoner ble $\{z_2, z_2, z_4, z_1\}$. Hvilke er out-of-bag?
2. En kollega sier: «Jeg kjørte Random Forest med 2000 trær i stedet for 500, og fikk dårligere testfeil. Modellen overtilpasser.» Hva er den mest sannsynlige forklaringen på det han observerte? $\text{test feil}_{500} < \text{test feil}_{2000}$
3. Du har $p = 40$ prediktorer der X_1 er en svært sterk prediktor og 20 av de øvrige er sterkt korrelerte med hverandre. Du kjører bagging og får omtrent samme testfeil som for ett enkelt tre. Forklar hvorfor, og foreslå hva du gjør videre — inkludert en konkret verdi for m .
4. I boosting med $\lambda = 0,01$ og $B = 1000$ er testfeilen fortsatt fallende når du stopper. Hva gjør du? Hva skjer hvis du i stedet setter $\lambda = 0,5$ og lar $B = 1000$ stå?
5. Til diskusjon: En bank vil bruke en modell til å avslå lånesøknader og må kunne begrunne hvert avslag overfor kunden. Random Forest gir 4 prosentpoeng lavere feilrate enn ett tre. Hva anbefaler dere, og hvorfor?



Fem setninger å ta med videre

1. Gjennomsnitt reduserer varians — bootstrap gir oss de mange datasettene vi trenger for å ta det.
2. Bagging estimerer B dype trær på bootstrap-utvalg og midler dem; bias bevares, varians faller.
3. Hver observasjon er *out of bag* i ca. $1/3$ av trærne, og gir oss OOB-feilen gratis.
4. Random Forest er bagging med $m < p$: ved å utestenge den sterke prediktoren fra de fleste splittene senkes korrelasjonen mellom trærne, og gjennomsnittet blir mer stabilt.
5. Boosting bygger små trær sekvensielt på residualene, styrt av B , λ og d — og lærer sakte med vilje.

tuning parametrene!

Viktig: Neste gang er det kapittel 9 — Support Vector Machines

Vi går fra å dele rommet i bokser til å finne én optimal skillelinje. Nøkkelordene er *margin*, *støttevektorer* og *kjernefunksjoner* — fra trærnes logikk til geometrisk optimering.

Takk for i dag. Husk å begynne på prosjektet! :-)